

XENONnT Event Viewer

软件设计、实现与优化完整文档

2026 年 5 月

目录

1	项目概述	3
1.1	背景	3
1.2	核心功能	3
1.3	技术栈	3
2	架构设计	4
2.1	分层架构	4
2.2	数据流	4
2.3	数据类型支持	5
2.4	NPZ 数据格式	5
3	关键实现细节	6
3.1	画布管理：彻底解决残影	6
3.1.1	问题演化	6
3.2	交互系统	7
3.2.1	Peaks 点击命中测试	7
3.2.2	图例交互切换	7
3.2.3	滚轮缩放	8
3.2.4	PMT ID 悬停显示	8
3.3	Peak 列表排序	8
3.4	键盘快捷键	8
3.5	导出功能	9
3.5.1	演进	9
3.6	PMT 击中模式渲染	9
3.6.1	圆圈半径修复	9

3.6.2	颜色条 (Colorbar) 修复	9
3.6.3	PMT 点大小调整	9
3.7	字体与样式优化	9
3.8	波形线条粗细优化	10
3.9	波形颜色方案	10
4	数据探索	10
4.1	数据源调查	10
4.1.1	DALI 探测	10
4.1.2	Midway3 探测	11
4.1.3	数据层级	11
4.2	真实数据提取	11
4.3	传输问题	12
5	测试与验证	12
5.1	自动化测试套件	12
5.2	端到端验证	12
6	PyInstaller 打包	12
6.1	构建参数	12
6.2	版本号	13
6.3	触发的 CI 构建	13
7	完整踩坑记录	14
8	目前已知限制	15
9	参考文献	15

1 项目概述

1.1 背景

XENONnT 是位于意大利 Gran Sasso 地下实验室的暗物质探测实验。实验产生大量波形数据,研究人员需要交互式浏览工具来查看事件波形和 PMT 击中模式。EventViewer 是一个跨平台桌面应用,支持离线查看预提取的 NPZ 数据包。

1.2 核心功能

- **事件浏览**: 左侧面板列出所有事件,支持 S1/S2 面积筛选、事件编号搜索
- **Peak 列表**: 可排序表格,显示 Type、Area (PE)、Width (ns)、Rise (ns)、Time (ns)
- **主峰标注**: Main S1/Main S2 在列表中加粗显示
- **三层波形显示**: Top PMT (蓝 #2196F3) + Bottom PMT (绿 #4CAF50) + Total (红 #F44336)
- **交互式 Peaks Zoom**: 点击波形或表行进入单峰缩放模式
- **图例交互切换**: 点击图例文字切换波形层显示/隐藏
- **滚轮缩放**: 鼠标滚轮在波形面板上缩放
- **页面缩放**: +/- 按钮或 Ctrl+ 滚轮调整全局页面缩放百分比
- **PMT ID 悬停**: 鼠标悬停在 PMT 散点图上显示通道 ID
- **PDF/PNG 导出**: Ctrl+S 直接保存到桌面
- **Run Selector**: 下拉菜单自动扫描切换内嵌 NPZ 数据文件
- **多数据格式支持**: NPZ (peak_basics) 模型波形, NPZ (real peaks) 真实波形

1.3 技术栈

- Python 3.9+
- PySide6 (Qt GUI)
- Matplotlib (科学绘图)
- NumPy (数据处理)
- Strax/Straxen (可选,用于直接数据访问)
- PyInstaller (打包)

2 架构设计

2.1 分层架构

```
run_app.py                # 入口点, 命令行参数解析, 自动加载 NPZ
+-- event_viewer_app/     # Qt GUI 层
|   +-- main_window.py    # MainWindow 主窗口
|   |   +-- PeakListWidget # Peaks 列表组件
|   |   +-- Run Selector (QComboBox) # 运行选择器
|   +-- event_browser.py   # 事件列表, 搜索, 筛选
|   +-- event_canvas.py    # Matplotlib 画布 + 鼠标/键盘交互
|   +-- data_manager.py    # 数据加载与缓存管理
|   +-- qt_compat.py       # PySide6/PySide2 兼容层
+-- event_plotter/        # 绘图引擎
|   +-- plotter.py         # 核心绘图函数
|   |   +-- plot_event_full() # 事件全览图
|   |   +-- plot_peak_zoom()  # 单峰缩放图
|   |   +-- plot_pmt_hit_pattern() # PMT 击中模式
|   |   +-- plot_peaks()      # 全部 peaks 堆叠
|   |   +-- _draw_3layer_waveform() # 三层波形绘制
|   +-- io.py              # 数据 I/O, PMT 几何加载
|   +-- style.py           # 样式配置, 颜色调色板
+-- scripts/              # CLI 工具
|   +-- extract_event_bundle.py # 从 strax 提取 NPZ
|   +-- plot_events.py        # 批量绘图
|   +-- build_mac.sh         # macOS PyInstaller 构建
+-- dali_probe/           # DALI 数据探测
|   +-- extract_peaks_bundle.py # DALI real peaks 提取
|   +-- report.md            # DALI 探测报告
+-- debug_reports/        # 调试报告
    +-- 00_SUMMARY.md       # 总结
    +-- 01_static_analysis.txt # 静态分析
    +-- 02_data_pipeline_tests.txt # 数据管线测试
    +-- 03_rendering_edge_cases.txt # 渲染边界测试
    +-- 04_ui_simulation_tests.txt # UI 模拟测试
```

2.2 数据流

1. NPZ 文件 → `DataManager.open_npz_bundle()`

2. 加载 `events[]`, `peaks_list[]`, `eac_list[]`, `pmt_positions`, `to_pe`
3. `EventBrowser` 填充事件列表, 自动选中第一个事件
4. 用户点击事件 → `MainWindow._on_event_selected()`
5. → `plot_event_full()` → `EventCanvas` 渲染
6. Peak 点击 → `EventCanvas` 命中测试
7. → `peak_clicked` Signal → `MainWindow._on_peak_clicked()`
8. → `plot_peak_zoom()` → 三面板视图

2.3 数据类型支持

数据源	波形类型	PMT Pattern	data_top	来源
NPZ peak_basics	模型脉冲 (双指数)	EAC 按比例缩放	无	Midway3 processed
NPZ real peaks	真实 data 采样	真实 area_per_channel	有	DALI
Strax (cutax)	真实 data	真实 area_per_channel	有	在线/离线 context

表 1: 数据源对比

2.4 NPZ 数据格式

NPZ bundle 包含以下键:

- `events`: 事件信息结构数组
- `peaks_list`: 每个事件的 peaks 列表 (object dtype)
- `eac_list`: 每个事件的事件_area_per_channel (可选)
- `pmt_x`, `pmt_y`, `pmt_array`, `pmt_i`: PMT 位置 (分列存储)
- `to_pe`: PMT 增益因子
- `run_id`: 运行编号
- `waveform_source`: “real_peaks” 或 “model” (新增字段)

3 关键实现细节

3.1 画布管理：彻底解决残影

3.1.1 问题演化

1. **初版 (fig.clear()):** 重用 Figure, 调用 fig.clear()。切换事件/peaks 时旧内容残留, 出现”鬼影”重叠。
2. **fig.clf() 版:** 改用 fig.clf() 完全清除。仍然有边缘彩色残影, 尤其在滚动时。
3. **delaxes 版:** 逐轴删除 fig.delaxes(ax)。残留依然存在。
4. **QScrollArea 版:** 用 QScrollArea 包裹画布, setWidgetResizable(False)。边缘出现彩色像素点, 页面显示”乱码”。
5. **最终方案:** 每次 clear() 创建全新的 Figure + FigureCanvasQTAagg, 旧画布 deleteLater()。彻底清除所有残留。

Listing 1: EventCanvas.clear() 最终实现

```
def clear(self):
    if self._hover_annot:
        try: self._hover_annot.remove()
        except Exception: pass
        self._hover_annot = None
    # Replace entire figure to eliminate ghosting
    old_canvas = self._canvas
    self._fig = Figure(figsize=(16, 10), facecolor="white", dpi=self._base_dpi)
    self._canvas = FigureCanvasQTAagg(self._fig)
    self._canvas.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Expanding)
    self._canvas.mpl_connect('button_press_event', self._on_click)
    self._canvas.mpl_connect('scroll_event', self._on_scroll)
    self._canvas.mpl_connect('motion_notify_event', self._on_hover)
    layout = self.layout()
    if layout:
        idx = layout.indexOf(old_canvas)
        if idx >= 0: layout.insertWidget(idx, self._canvas)
        else: layout.addWidget(self._canvas)
    old_canvas.setParent(None)
    old_canvas.deleteLater()
    if self._toolbar is not None:
        self._toolbar.canvas = self._canvas # Fix save button
```

```
self._update_canvas_size()
```

3.2 交互系统

3.2.1 Peaks 点击命中测试

点击波形区域后，通过邻近搜索找到最近的 peak：

```
def _on_click(self, event):
    regions = getattr(event.inaxes, '_peak_regions', None)
    if not regions: return
    x = event.xdata
    MAX_CLICK_DIST = 0.0005 # 500 microseconds tolerance
    best, best_d = None, MAX_CLICK_DIST
    for r in regions:
        cx = r.get('center_x', (r['x_start'] + r['x_end']) / 2)
        d = abs(x - cx)
        if d < best_d: best_d = d; best = r
    if best is not None:
        self.peak_clicked.emit(best['index'])
```

每个 peak 在绘制时存储其时间区域到 `ax._peak_regions` 列表。最初使用严格区间匹配 (`x_start <= x <= x_end`)，但窄峰（微秒级）在屏幕上不到 1 像素宽，无法有效点击。改为邻近搜索（500 s tolerance），同时保留区间匹配作为备选。

3.2.2 图例交互切换

三层波形的图例文字可点击切换每层的显示。实现方式：在 `_on_click` 中检测点击是否命中图例文字，若命中则切换对应层的可见性。

```
def _check_legend_click(self, event):
    state = getattr(event.inaxes, '_legend_state', None)
    artists = getattr(event.inaxes, '_legend_artists', None)
    leg = event.inaxes.get_legend()
    for txt in leg.get_texts():
        if txt.contains(event)[0]:
            for key in state:
                if key in txt.get_text().lower():
                    state[key] = not state[key]
                    artists.get(key).set_visible(state[key])
            return
```

3.2.3 滚轮缩放

鼠标滚轮在波形面板上缩放该面板的 x/y 轴范围。Page 缩放通过 Ctrl+ 滚轮或 +/- 按钮调整 Figure DPI。

```
def _on_scroll(self, event):
    if event.inaxes is None: return
    s = 0.8 if event.button == 'up' else 1.25
    cx, cy = event.xdata, event.ydata
    xmin, xmax = ax.get_xlim()
    ymin, ymax = ax.get_ylim()
    ax.set_xlim(cx - (cx - xmin) * s, cx + (xmax - cx) * s)
    ax.set_ylim(cy - (cy - ymin) * s, cy + (ymax - cy) * s)
```

3.2.4 PMT ID 悬停显示

`motion_notify_event` 检测鼠标位置是否在 PMT 散点图的任一数据点上。若命中，显示黄色标注框显示 PMT 通道 ID。

3.3 Peak 列表排序

QTableWidget 默认按字符串排序，导致数值列（Area、Width 等）排序错误（如“9” > “100”）。

解决方案：自定义 `NumericItem` 类，重载 `__lt__` 方法实现数值比较。同时通过 `Qt.UserRole` 在每行保存原始 peak 索引，确保排序后点击正确的 peak。

```
class NumericItem(QTableWidgetItem):
    def __lt__(self, other):
        try: return float(self.text()) < float(other.text())
        except: return super().__lt__(other)
```

3.4 键盘快捷键

方向键导航的难点：QListWidget 和 QTableWidget 会消费 Up/Down 按键。解决方案：

1. 使用 Left/Right 而非 Up/Down 切换事件
2. 使用 QShortcut 并指定 WindowShortcut 上下文：
`QShortcut(QKeySequence("Left"), self).activated.connect(...)`
3. Escape 键同样使用 QShortcut

3.5 导出功能

3.5.1 演进

1. **QFileDialog 版**: 使用 `QFileDialog.getSaveFileName()` 弹出保存对话框。在 PyInstaller 打包的 .app 中对话框可能无法弹出 (z-order 问题)。
2. **桌面直存版**: 跳过对话框, 直接保存到桌面, 自动递增文件名避免覆盖。Ctrl+S 触发。
3. **工具栏保存按钮**: NavigationToolbar2QT 自带保存按钮, 但画布替换后工具栏仍引用旧 canvas。修复: `self._toolbar.canvas = self._canvas`
4. **PDF 后端缺失**: PyInstaller 不包含 `matplotlib.backends.backend_pdf`。修复: 添加 `--hidden-import` 到构建参数。

3.6 PMT 击中模式渲染

3.6.1 圆圈半径修复

硬编码 TPC 半径 47.9cm 远小于实际 PMT 阵列最大半径 65.6cm。初次实现时 252/494 个 PMT 在圆圈外。

修复: 从 PMT 位置数据自动计算边界圆半径:

```
pmt_r = np.sqrt(pmt_positions["x"]**2 + pmt_positions["y"]**2).max()
r_bound = pmt_r * 1.02
```

3.6.2 颜色条 (Colorbar) 修复

使用 `plt.colorbar()` 在画布替换后颜色条附加到错误的 Figure。全部替换为 `ax.figure.colorbar()`。

3.6.3 PMT 点大小调整

经历了多次迭代: $50 \rightarrow 25 \rightarrow 12 \rightarrow 25 \rightarrow 60$ (最后一版加倍)。最终值: 事件全览 120, Peaks Zoom 120。

3.7 字体与样式优化

经历了多轮调整:

版本	基础字号	图例字号	坐标轴线宽
----	------	------	-------

初版	9pt	+0 (9pt)	1.5
第一次优化	11pt	+3 (14pt)	2.0
第二次优化	11pt	+6 (17pt)	2.0
最终版	11pt	+14 (25pt)	2.0

表 2: 字体演化

图例边框在最终版中完全移除 (`legend.frameon = False`)。

标题从 `fontsize=_rsize+1` 增加到 +3, `suptitle` 增加到 +4。

3.8 波形线条粗细优化

组件	初版	第一次优化	最终版
事件全览 Top/Bot	0.25	0.75	0.75
事件全览 Total	0.35	1.05	1.05
Peaks Zoom Top/Bot	0.8	2.4	2.4
Peaks Zoom Total	1.5	4.5	4.5

表 3: 线宽演化

所有线宽乘以 3 倍。数据模式（有波形采样）下的步进线宽从 [0.45, 0.45, 0.8] 增加到 [1.35, 1.35, 2.4]。

3.9 波形颜色方案

从柔和的 Nature 期刊配色（紫色/橙色/灰色）改为鲜艳的蓝绿红：

- Top PMT: #2196F3 (Material Blue) —原为 Violet (#9A4D8E)
- Bottom PMT: #4CAF50 (Material Green) —原为 Orange (#E67E22)
- Total: #F44336 (Material Red) —原为 peak 类型色

4 数据探索

4.1 数据源调查

4.1.1 DALI 探测

DALI (`ssh dali`) 是 XENON 的专用数据节点。探测发现：

- `/dali/lgrandi/xenonnt/raw/`: 仅有 `raw_records_aqmon` (监控数据), 无 TPC 物理 `raw_records`
- `/dali/lgrandi/xenonnt/processed/`: 包含带真实波形的 `peaks` (043864, 044116 等)
- `peaks` dtype 包含: `data` (200 样本, PE/sample), `data_top` (top 阵列波形), `area_per_channel` (494 PMT)

4.1.2 Midway3 探测

Midway3 (`ssh midway3`) 是主要计算集群:

- `/project/lgrandi/xenonnt/processed/`: 2193 runs 的 `peak_basics`
- `/project2/lgrandi/xenonnt/processed/`: 更多 runs
- `straxen.contexts.xenonnt()`: 查询了 73,925 个 run, 0 个有 `raw_records` 或 `records` 在本地存储
- 023756 仅有 `raw_records_aqmon`, 无 TPC `raw_records`
- 280 runs 有 `peaklets` (包含 `data`, `area_per_channel`)

4.1.3 数据层级

<code>raw_records</code>	<code>→</code>	<code>records</code>	<code>→</code>	<code>peaklets</code>	<code>→</code>	<code>peaks</code>	<code>→</code>	<code>peak_basics</code>
(raw)		(ADC)		(per-PMT)		(merged)		(metadata)

Midway3 主要存储: `peak_basics` (元数据, 轻量), `event_info`, `event_area_per_channel`。
DALI 额外存储: `peaklets`, `peaks` (带真实波形采样)。

4.2 真实数据提取

从 DALI 提取 real peaks 的流程:

1. SSH 到 DALI
2. source XENON 环境: `/cvmfs/xenon.opensciencegrid.org/releases/nT/e17.2025.07.2/setu`
3. 运行 `dali_probe/extract_peaks_bundle.py --run 043864 --n 200`
4. 生成 NPZ: 9.1MB, 200 events, 3566 peaks
5. 从 Midway3 拉回本地 (`rsync`, 反复重试)

4.3 传输问题

SCP 从 Midway3 传输大文件时常中断（9.1MB 文件经过多次尝试才完整拉回）。解决：

- 使用 `rsync` 替代 `scp`（支持断点续传）
- 多次重试
- 提取更小的样本（5-50 events）用于快速验证

5 测试与验证

5.1 自动化测试套件

独立调试报告位于 `debug_reports/`：

报告	测试数	结果	说明
01_static_analysis	11 files	2 问题发现	导入、打印语句、硬编码路径
02_data_pipeline	9 项	全部通过	模式切换、缓存、索引
03_rendering_edges	8 项	全部通过	空 peaks、内存、DPI 极端值
04_ui_simulation	8 项	全部通过	排序、UserRole、主峰标记

表 4: 调试报告

5.2 端到端验证

- 023756 NPZ (model): 50 events, 69 peaks/event —全部加载
- 043864 real peaks (5ev): 5 events, 真实波形—通过
- 043864 real peaks (200ev): 200 events, 3566 peaks —通过
- 交叉切换：NPZ $\rightarrow$ real peaks $\rightarrow$ NPZ —缓存正确清理
- 20 次事件切换无渲染错误
- 50 次 peak zoom 无 area 不一致
- 100 次渲染无内存泄漏

6 PyInstaller 打包

6.1 构建参数

```
pyinstaller \
--onedir --windowed \
--name XENONnT-EventViewer \
--add-data "scripts/output/events_run_023756.npz:." \
--hidden-import PySide6.QtCore \
--hidden-import PySide6.QtGui \
--hidden-import PySide6.QtWidgets \
--hidden-import matplotlib.backends.backend_qtagg \
--hidden-import matplotlib.backends.backend_pdf \
--hidden-import matplotlib.backends.backend_agg \
--collect-submodules numpy \
--collect-data matplotlib \
--exclude-module tkinter \
--exclude-module PyQt5 \
--exclude-module scipy \
--exclude-module pandas \
--distpath dist --workpath /tmp/pyinstaller_mac \
run_app.py
```

6.2 版本号

构建脚本通过 PlistBuddy 注入版本号到 Info.plist:

```
/usr/libexec/PlistBuddy -c "Set_:CFBundleShortVersionString_2.0.0" \
"dist/XENONnT-EventViewer.app/Contents/Info.plist"
```

6.3 触发的 CI 构建

GitHub Actions (`.github/workflows/build.yml`) 在推送 `v*` 标签时触发:

- build-linux: ubuntu-latest, PyInstaller → .tar.gz
- build-windows: windows-latest, PyInstaller → .zip
- build-macos-arm64: macos-latest, PyInstaller → .dmg
- build-macos-x64: macos-13, PyInstaller → .dmg

7 完整踩坑记录

1. QScrollArea 残影

尝试方案: `setWidgetResizable(False/True)`、`setFixedSize`、`repaint()`、`draw()`
vs `draw_idle()`、scroll widget 嵌套。

最终方案: 完全放弃 `QScrollArea`, 每次 `clear()` 创建新 `Figure+Canvas`。

2. 工具栏保存按钮失效

原因: 画布替换后 `NavigationToolBar2QT` 仍引用旧 `canvas`。

修复: `self._toolbar.canvas = self._canvas`。

3. PDF 导出缺少 backend

原因: `PyInstaller` 不包含 `matplotlib.backends.backend_pdf`。

修复: 添加 `--hidden-import` 到构建脚本。

4. Peak 列表排序错位

原因: `QTableWidget` 字符串排序。

修复: `NumericItem + Qt.UserRole` 存储原始索引。

5. macOS .app 打开终端

原因: `PyInstaller` 单文件模式在 macOS 上打开终端。

修复: `--onedir --windowed`。

6. strax 导入错误

原因: `strax` 是可选依赖但被列在 `hidden_imports` 中。

修复: 从 `hidden_imports` 移除 `strax`。

7. PMT 圆圈不匹配

原因: 硬编码 47.9cm (TPC 半径) vs 实际 PMT 最大 65.6cm。

修复: 从 PMT 位置自动计算。

8. plt.colorbar 引用错误 Figure

原因: `plt.colorbar()` 使用 `pyplot` 状态机, 画布替换后引用错位。

修复: 全部改为 `ax.figure.colorbar()`。

9. 内联注释破坏语法

原因: 全局查找替换颜色时, 注释被插入函数调用中间。

样例: `color="#2196F3" # bright blue, alpha_fill=0.2` ← 注释吞掉了后续参数。

修复: 逐行检查并移除函数参数中的内联注释。

10. DALI/Midway3 SSH 传输不稳定

原因: SCP 对大文件 (9MB+) 经常断连。

修复: rsync + 断点续传 + 多次重试。

11. 三层波形 PMT Pattern 全为零

原因: NPZ 模式下 EAC 加载被 strax 模式守卫阻塞。

修复: 移除 `if self._dm.mode == "strax"` 守卫, NPZ 模式也加载 EAC。

12. 事件波形空白 (Build 不含 Codex 修改)

原因: App 在 Codex 修改源码后构建, 但源码修改未写入构建缓存。

修复: 清除 PyInstaller 缓存, 确认源文件时间戳, 重新构建。

13. 滚轮缩放导致”乱码”

原因: Trackpad 惯性滚动产生大量 scroll 事件, `draw_idle()` 渲染跟不上。

修复: 改为同步 `draw()` 渲染。

14. 切换事件/peaks 时旧波形残留

原因: matplotlib 的 `fig.clear()` 和 `fig.clf()` 不会立即更新 Qt widget。

修复: 最终采用全新 Figure+Canvas 方案。

15. 左侧面板不可调整大小

原因: `QSplitter stretchFactor(0,0) + setMaximumWidth(320)` 锁定左侧面板。

修复: `stretchFactor(0,1)`, `maximumWidth` 改为 500。

8 目前已知限制

- 023756 无 TPC raw_records, 波形为模型生成
- real peaks 数据需要从 DALI 手动提取并传输
- Strax 模式在主线程同步加载, 大 run 可能卡住界面
- Windows/Linux 版本需要 CI 或本地构建, 无法在 macOS 上交叉编译
- .app 使用 ad-hoc 签名, 分发时可能触发 Gatekeeper 警告
- Run Selector 扫描路径在打包后可能找不到外部 NPZ 文件

9 参考文献

- XENONnT Straxen 文档: <https://straxen.readthedocs.io>
- PySide6 文档: <https://doc.qt.io/qtforpython-6/>
- Matplotlib 文档: <https://matplotlib.org/stable/>

- PyInstaller 手册: <https://pyinstaller.org/en/stable/>